

Chapter 39 — Interval Dynamic Programming

0. What You Will Learn in This Chapter

Interval Dynamic Programming is used when the state represents a range:

`dp[l][r]`

or:

`dp[i][j]`

where `l` and `r` are the left and right boundaries of a subarray, substring, or interval.

Interval DP is common when the problem asks you to:

- remove elements in some order
- burst balloons
- merge stones
- cut sticks
- triangulate polygons
- choose an operation inside a range
- optimize cost over subarrays
- solve palindrome range problems
- split an interval into smaller intervals

By the end of this chapter, you should understand:

- What interval DP is
- When to define `dp[l][r]`
- Why interval DP often chooses the **last operation**
- How to fill by interval length
- How split-point transitions work
- How to solve min-cost range partition problems
- How to solve max-score range removal problems
- How interval DP differs from normal 2D DP
- How to avoid boundary mistakes

Core problems covered in this chapter:

1. Burst Balloons
 2. Minimum Cost to Cut a Stick
 3. Matrix Chain Multiplication
 4. Minimum Score Triangulation of Polygon
 5. Stone Game
 6. Stone Game II, preview
 7. Stone Game III, preview
 8. Stone Game VII
 9. Merge Stones
 10. Palindrome Partitioning II, interval relation
 11. Longest Palindromic Subsequence, interval relation
 12. Strange Printer
 13. Remove Boxes, preview
 14. Scramble String, preview
 15. Boolean Parenthesization, preview
-

1. What Is Interval DP?

Interval DP solves problems where the answer for a range depends on smaller ranges.

The state usually looks like:

$dp[l][r]$ = best answer for subarray/string/range from l to r

Example:

Longest Palindromic Subsequence:

$dp[l][r]$ = length of longest palindromic subsequence in $s[l..r]$

Burst Balloons:

$dp[l][r]$ = maximum coins from bursting balloons between l and r

Minimum Cost to Cut a Stick:

$dp[l][r]$ = minimum cost to fully cut the segment between $cuts[l]$ and $cuts[r]$

2. Why Interval DP Exists

Some problems cannot be solved by a simple left-to-right DP because operations inside a range affect both sides.

Example:

Burst Balloons.

If you burst a balloon first, its neighbors change.

That makes the next state hard to describe.

But if you choose the balloon that is burst **last** inside an interval, its outside neighbors are fixed.

This is a key interval DP trick:

Instead of choosing the first operation, choose the last operation.

3. Core Interval DP Idea

Most interval DP problems follow this structure:

$dp[l][r]$ = best answer for interval $[l, r]$

Then try a split point k :

$dp[l][r]$ = best over k of:

$$dp[l][k] + dp[k][r] + \text{cost}(l, k, r)$$

or:

$dp[l][r]$ = best over k of:

$$dp[l][k - 1] + dp[k + 1][r] + \text{cost}(l, k, r)$$

The exact boundary depends on the problem.

4. Interval DP vs Grid DP

Both may use a 2D table, but they are different.

State	$dp[r][c]$ cell	$dp[l][r]$ range
Fill order	row/column order	by interval length
Dependency	top/left/diagonal	smaller intervals
Common transition	move into cell	split interval
Example	Unique Paths	Burst Balloons

Grid DP:

$dp[r][c]$ depends on nearby cells

Interval DP:

$dp[l][r]$ depends on shorter intervals inside $[l, r]$

5. When to Use Interval DP

Use interval DP when:

5.1 The Problem Is About a Range

Examples:

- subarray
- substring
- segment
- stick interval
- polygon interval

5.2 You Can Split the Range

Examples:

- choose a cut point
- choose a last balloon
- choose a triangle root
- choose a merge point

5.3 Operations Change Neighbor Relationships

Examples:

- Burst Balloons
- Remove Boxes
- Strange Printer

5.4 Cost Depends on Interval Boundaries

Examples:

- Minimum Cost to Cut a Stick
- Matrix Chain Multiplication
- Merge Stones

5.5 Palindrome or Substring Range

Examples:

- Longest Palindromic Subsequence
 - Palindromic Substrings
 - Strange Printer
-

6. When NOT to Use Interval DP

Do not use interval DP when:

6.1 State Is Only a Prefix

Use 1D or standard 2D DP.

Example:

$dp[i]$ = best answer for first i elements

6.2 You Need Shortest Path

Use BFS or Dijkstra.

6.3 You Need Independent Item Choice With Capacity

Use knapsack DP.

6.4 You Need All Solutions

Use backtracking.

6.5 Range Query Updates Are Dynamic

Use segment tree, Fenwick tree, or balanced tree.

7. Mental Models

7.1 Solve Smaller Intervals First

If $dp[l][r]$ depends on smaller ranges, fill by length:

length = 1
length = 2
length = 3
...

7.2 Choose the Last Operation

This is often the key trick.

Examples:

- Last balloon burst
- Last cut made
- Last multiplication split
- Last triangle chosen

Choosing the last operation often fixes the outer boundaries.

7.3 Split the Interval

Try every possible split point:

$[l \dots k]$ and $[k+1 \dots r]$

or:

$[l \dots k]$ and $[k \dots r]$

7.4 Boundary Padding

Some problems become easier by adding fake boundaries.

Example:

Burst Balloons adds 1 at both ends.

```
nums = [3,1,5,8]
```

```
padded = [1,3,1,5,8,1]
```

8. Pattern Recognition Checklist

Ask:

16. Does the problem involve subarrays/substrings/ranges?
17. Can I define `dp[l][r]`?
18. Does solving a range require solving smaller ranges?
19. Is there a split point `k`?
20. Does cost depend on left and right boundaries?
21. Does operation order matter?
22. Is choosing the first operation hard?
23. Would choosing the last operation simplify boundaries?
24. Can I fill by interval length?
25. Is there a need to add sentinel boundaries?

9. C++ Tools Needed

9.1 2D Vector

```
vector<vector<int>> dp(n, vector<int>(n, 0));
```

9.2 INF for Minimum Cost

```
const int INF = 1e9;  
vector<vector<int>> dp(n, vector<int>(n, INF));
```

9.3 Long Long for Large Costs

```
vector<vector<long long>> dp(n, vector<long long>(n, 0));
```

9.4 Sort Cuts / Boundaries

```
sort(cuts.begin(), cuts.end());
```

9.5 Prefix Sum for Range Sum

```
vector<int> prefix(n + 1, 0);  
  
for (int i = 0; i < n; i++) {  
    prefix[i + 1] = prefix[i] + nums[i];  
}  
  
int sum = prefix[r + 1] - prefix[l];
```

10. Core Templates

Template 1 — Basic Interval DP by Length

```
for (int len = 1; len <= n; len++) {
    for (int l = 0; l + len - 1 < n; l++) {
        int r = l + len - 1;

        // compute dp[l][r] from smaller intervals
    }
}
```

Template 2 — Split Interval Min Cost

```
for (int len = 2; len <= n; len++) {
    for (int l = 0; l + len - 1 < n; l++) {
        int r = l + len - 1;

        dp[l][r] = INF;

        for (int k = l; k < r; k++) {
            dp[l][r] = min(dp[l][r],
                           dp[l][k] + dp[k + 1][r] + cost(l, r));
        }
    }
}
```

Template 3 — Open Interval DP

Used when boundaries are not included in operations.

```
for (int len = 2; len < n; len++) {
    for (int l = 0; l + len < n; l++) {
        int r = l + len;

        for (int k = l + 1; k < r; k++) {
            dp[l][r] = max(dp[l][r],
                           dp[l][k] + dp[k][r] + cost(l, k, r));
        }
    }
}
```

Common in:

- Burst Balloons
- Minimum Cost to Cut a Stick

Template 4 — Palindrome Range DP

```
for (int len = 1; len <= n; len++) {
    for (int l = 0; l + len - 1 < n; l++) {
        int r = l + len - 1;

        if (s[l] == s[r]) {
            dp[l][r] = 2 + dp[l + 1][r - 1];
        } else {
            dp[l][r] = max(dp[l + 1][r], dp[l][r - 1]);
        }
    }
}
```

Template 5 — Game Interval DP

```
for (int len = 1; len <= n; len++) {
    for (int l = 0; l + len - 1 < n; l++) {
        int r = l + len - 1;

        dp[l][r] = max(nums[l] - dp[l + 1][r],
                        nums[r] - dp[l][r - 1]);
    }
}
```

Here:

$dp[l][r]$ = maximum score difference current player can achieve

11. Major Interval DP Variations

Choose last item	$dp[l][r]$ open interval	Burst Balloons
Cut interval	$dp[l][r]$ min cut cost	Minimum Cost to Cut Stick
Split chain	$dp[l][r]$ min multiplication cost	Matrix Chain
Polygon split	$dp[l][r]$ min score	Triangulation
Game range	$dp[l][r]$ score difference	Stone Game

Variation	State	Example
Merge range	$dp[l][r]$ min merge cost	Merge Stones
Palindrome range	$dp[l][r]$ palindrome answer	LPS
Print range	$dp[l][r]$ min turns	Strange Printer

12. Full Problem Walkthrough 1 — LeetCode 312: Burst Balloons

12.1 Problem Summary

Given balloons with numbers.

When bursting balloon i , gain:

$$\text{nums}[\text{left}] * \text{nums}[i] * \text{nums}[\text{right}]$$

After bursting, neighbors become adjacent.

Return maximum coins.

12.2 Why This Is Interval DP

If we choose the first balloon to burst, neighbors change unpredictably.

Instead choose the **last balloon** burst inside an interval.

Then left and right boundaries are fixed.

12.3 Add Boundaries

Original:

[3,1,5,8]

Padded:

[1,3,1,5,8,1]

12.4 State

$dp[l][r]$ = max coins from bursting balloons strictly between l and r

So l and r are boundaries, not burst.

12.5 Transition

Choose last balloon k between l and r :

$$dp[l][r] = \max(dp[l][k] + \text{nums}[l] * \text{nums}[k] * \text{nums}[r] + dp[k][r])$$

12.6 C++ Code

```
class Solution {
public:
    int maxCoins(vector<int>& nums) {
```

```

vector<int> arr;

arr.push_back(1);

for (int x : nums) {
    arr.push_back(x);
}

arr.push_back(1);

int n = arr.size();

vector<vector<int>> dp(n, vector<int>(n, 0));

for (int len = 2; len < n; len++) {
    for (int l = 0; l + len < n; l++) {
        int r = l + len;

        for (int k = l + 1; k < r; k++) {
            int coins = dp[l][k] + arr[l] * arr[k] * arr[r] + dp[k][r];

            dp[l][r] = max(dp[l][r], coins);
        }
    }
}

return dp[0][n - 1];
};

```

12.7 Complexity

Time:

$O(n^3)$

Space:

$O(n^2)$

13. Full Problem Walkthrough 2 — LeetCode 1547: Minimum Cost to Cut a Stick

13.1 Problem Summary

You have a stick of length n .

You are given cut positions.

Each cut costs the current stick length.

Return minimum total cost to perform all cuts.

13.2 Why This Is Interval DP

A cut splits a stick interval into two smaller intervals.

Cost depends on current interval length.

13.3 Add Boundaries

Add:

0

n

to cuts.

Sort them.

13.4 State

$dp[l][r]$ = minimum cost to cut stick segment from cuts[l] to cuts[r]

Only cuts strictly between l and r are performed.

13.5 Transition

Choose first cut k inside (l, r) :

$dp[l][r] = \min(dp[l][k] + dp[k][r] + \text{cuts}[r] - \text{cuts}[l])$

13.6 C++ Code

```
class Solution {
public:
    int minCost(int n, vector<int>& cuts) {
        cuts.push_back(0);
        cuts.push_back(n);

        sort(cuts.begin(), cuts.end());

        int m = cuts.size();

        vector<vector<int>> dp(m, vector<int>(m, 0));

        for (int len = 2; len < m; len++) {
            for (int l = 0; l + len < m; l++) {
                int r = l + len;

                dp[l][r] = 1e9;

                for (int k = l + 1; k < r; k++) {
```

```

        int cost = dp[l][k] + dp[k][r] + cuts[r] - cuts[l];

        dp[l][r] = min(dp[l][r], cost);
    }
}

return dp[0][m - 1];
}
};

```

13.7 Complexity

Time:

$O(c^3)$

where $c = \text{cuts.size()}$ after adding boundaries.

Space:

$O(c^2)$

14. Full Problem Walkthrough 3 — Matrix Chain Multiplication

14.1 Problem Summary

Given matrices:

$A_1, A_2, \dots, A_n$

Matrix A_i has dimensions:

$\text{dims}[i - 1] \times \text{dims}[i]$

Find minimum scalar multiplication cost.

14.2 Why This Is Interval DP

Choosing where to parenthesize splits the chain.

Example:

$(A_1 A_2) (A_3 A_4)$

Each split creates two smaller chains.

14.3 State

$\text{dp}[l][r]$ = minimum cost to multiply matrices $l..r$

14.4 Transition

Split at k :

```
dp[l][r] = min(
    dp[l][k] + dp[k + 1][r] + dims[l] * dims[k + 1] * dims[r + 1]
)
```

14.5 C++ Code

```
int matrixChainMultiplication(vector<int>& dims) {
    int n = dims.size() - 1;

    vector<vector<int>> dp(n, vector<int>(n, 0));

    for (int len = 2; len <= n; len++) {
        for (int l = 0; l + len - 1 < n; l++) {
            int r = l + len - 1;

            dp[l][r] = 1e9;

            for (int k = l; k < r; k++) {
                int cost = dp[l][k]
                    + dp[k + 1][r]
                    + dims[l] * dims[k + 1] * dims[r + 1];

                dp[l][r] = min(dp[l][r], cost);
            }
        }
    }

    return dp[0][n - 1];
}
```

14.6 Complexity

Time:

$O(n^3)$

Space:

$O(n^2)$

15. Full Problem Walkthrough 4 — LeetCode 1039: Minimum Score Triangulation of Polygon

15.1 Problem Summary

Given polygon vertices with values.
Triangulate polygon.
Score of triangle:

$\text{values}[i] * \text{values}[j] * \text{values}[k]$

Return minimum total score.

15.2 Why This Is Interval DP

Choosing a triangle (l, k, r) splits polygon interval into two smaller polygons.

15.3 State

$\text{dp}[l][r]$ = minimum triangulation score for polygon chain from l to r

15.4 Transition

Choose middle vertex k:

$\text{dp}[l][r] = \min(\text{dp}[l][k] + \text{dp}[k][r] + \text{values}[l] * \text{values}[k] * \text{values}[r])$

15.5 C++ Code

```
class Solution {
public:
    int minScoreTriangulation(vector<int>& values) {
        int n = values.size();

        vector<vector<int>> dp(n, vector<int>(n, 0));

        for (int len = 2; len < n; len++) {
            for (int l = 0; l + len < n; l++) {
                int r = l + len;

                dp[l][r] = 1e9;

                for (int k = l + 1; k < r; k++) {
                    int score = dp[l][k]
                        + dp[k][r]
                        + values[l] * values[k] * values[r];

                    dp[l][r] = min(dp[l][r], score);
                }
            }
        }

        return dp[0][n - 1];
    }
};
```

```
}  
};
```

15.6 Complexity

Time:

$O(n^3)$

Space:

$O(n^2)$

16. Full Problem Walkthrough 5 — LeetCode 877: Stone Game

16.1 Problem Summary

Two players take stones from either end of a row.

Both play optimally.

Return whether Alice wins.

16.2 Interval Game DP

Use score difference.

16.3 State

$dp[l][r]$ = maximum score difference current player can achieve over opponent
from piles[l..r]

16.4 Transition

Current player can take left:

$piles[l] - dp[l + 1][r]$

or right:

$piles[r] - dp[l][r - 1]$

So:

$dp[l][r] = \max(piles[l] - dp[l + 1][r],$
 $piles[r] - dp[l][r - 1])$

16.5 C++ Code

```
class Solution {  
public:  
    bool stoneGame(vector<int>& piles) {  
        int n = piles.size();  
  
        vector<vector<int>> dp(n, vector<int>(n, 0));  
  
        for (int i = 0; i < n; i++) {
```

```

        dp[i][i] = piles[i];
    }

    for (int len = 2; len <= n; len++) {
        for (int l = 0; l + len - 1 < n; l++) {
            int r = l + len - 1;

            dp[l][r] = max(piles[l] - dp[l + 1][r],
                           piles[r] - dp[l][r - 1]);
        }
    }

    return dp[0][n - 1] > 0;
}
};

```

16.6 Complexity

Time:

$O(n^2)$

Space:

$O(n^2)$

16.7 Note

For the original Stone Game constraints, Alice always wins mathematically.
 But the DP version teaches interval game logic.

17. Full Problem Walkthrough 6 — LeetCode 1690: Stone Game VII

17.1 Problem Summary

Players remove a stone from either end.

Score gained is the sum of remaining stones.

Return difference between Alice and Bob scores assuming optimal play.

17.2 State

$dp[l][r]$ = maximum score difference current player can get from stones[l..r]

17.3 Transition

If remove left, remaining sum is:

$sum(l + 1, r)$

Opponent then gains $dp[l + 1][r]$ advantage.

So current advantage:

$sum(l + 1, r) - dp[l + 1][r]$

Similarly remove right:

```
sum(l, r - 1) - dp[l][r - 1]
```

17.4 C++ Code

```
class Solution {
public:
    int stoneGameVII(vector<int>& stones) {
        int n = stones.size();

        vector<int> prefix(n + 1, 0);

        for (int i = 0; i < n; i++) {
            prefix[i + 1] = prefix[i] + stones[i];
        }

        auto rangeSum = [&](int l, int r) {
            return prefix[r + 1] - prefix[l];
        };

        vector<vector<int>> dp(n, vector<int>(n, 0));

        for (int len = 2; len <= n; len++) {
            for (int l = 0; l + len - 1 < n; l++) {
                int r = l + len - 1;

                int removeLeft = rangeSum(l + 1, r) - dp[l + 1][r];
                int removeRight = rangeSum(l, r - 1) - dp[l][r - 1];

                dp[l][r] = max(removeLeft, removeRight);
            }
        }

        return dp[0][n - 1];
    }
};
```

17.5 Complexity

Time:

$O(n^2)$

Space:

$O(n^2)$

18. Full Problem Walkthrough 7 — LeetCode 1000: Minimum Cost to Merge Stones

18.1 Problem Summary

You have stones in a row.

You can merge exactly k consecutive piles into one pile.

Cost of merging is the sum of those piles.

Return minimum cost to merge all stones into one pile.

If impossible, return -1 .

18.2 Why This Is Interval DP

Merging consecutive piles creates smaller intervals.

Cost depends on interval sum.

18.3 Feasibility Condition

Each merge reduces pile count by:

$k - 1$

To reduce n piles to 1:

$(n - 1) \% (k - 1) == 0$

must be true.

18.4 State

A full version uses:

$dp[l][r]$ = minimum cost to merge stones $[l..r]$ as much as possible

The transition jumps split points by $k - 1$.

18.5 C++ Code

```
class Solution {
public:
    int mergeStones(vector<int>& stones, int k) {
        int n = stones.size();

        if ((n - 1) % (k - 1) != 0) {
            return -1;
        }

        vector<int> prefix(n + 1, 0);

        for (int i = 0; i < n; i++) {
            prefix[i + 1] = prefix[i] + stones[i];
        }
    }
};
```

```

auto rangeSum = [&](int l, int r) {
    return prefix[r + 1] - prefix[l];
};

const int INF = 1e9;

vector<vector<int>> dp(n, vector<int>(n, 0));

for (int len = k; len <= n; len++) {
    for (int l = 0; l + len - 1 < n; l++) {
        int r = l + len - 1;

        dp[l][r] = INF;

        for (int mid = l; mid < r; mid += k - 1) {
            dp[l][r] = min(dp[l][r],
                           dp[l][mid] + dp[mid + 1][r]);
        }

        if ((len - 1) % (k - 1) == 0) {
            dp[l][r] += rangeSum(l, r);
        }
    }
}

return dp[0][n - 1];
};

```

18.6 Complexity

Time:

$O(n^3 / k)$

commonly described as $O(n^3)$.

Space:

$O(n^2)$

19. Full Problem Walkthrough 8 — LeetCode 664: Strange Printer

19.1 Problem Summary

A strange printer can print a sequence of the same character in one turn.
It can overwrite existing characters.

Return minimum turns to print string s.

19.2 Why This Is Interval DP

Printing a substring can be split into smaller substrings.

Matching characters can be printed in the same turn.

19.3 State

$dp[l][r]$ = minimum turns to print $s[l..r]$

19.4 Transition

Default:

$dp[l][r] = 1 + dp[l + 1][r]$

Print $s[l]$ separately.

If there exists k where:

$s[l] == s[k]$

then $s[l]$ can be printed together with $s[k]$.

$dp[l][r] = \min(dp[l][r], dp[l + 1][k - 1] + dp[k][r])$

19.5 C++ Code

```
class Solution {
public:
    int strangePrinter(string s) {
        int n = s.size();

        vector<vector<int>> dp(n, vector<int>(n, 0));

        for (int i = 0; i < n; i++) {
            dp[i][i] = 1;
        }

        for (int len = 2; len <= n; len++) {
            for (int l = 0; l + len - 1 < n; l++) {
                int r = l + len - 1;

                dp[l][r] = 1 + dp[l + 1][r];

                for (int k = l + 1; k <= r; k++) {
                    if (s[l] == s[k]) {
                        int middle = (k == l + 1) ? 0 : dp[l + 1][k - 1];

                        dp[l][r] = min(dp[l][r],
```

```

        middle + dp[k][r]);
    }
}
}
}

return dp[0][n - 1];
}
};

```

19.6 Complexity

Time:

$O(n^3)$

Space:

$O(n^2)$

20. Full Problem Walkthrough 9 — Longest Palindromic Subsequence Review

20.1 State

$dp[l][r]$ = longest palindromic subsequence length in $s[l..r]$

20.2 Transition

If ends match:

$dp[l][r] = 2 + dp[l + 1][r - 1]$

Otherwise:

$dp[l][r] = \max(dp[l + 1][r], dp[l][r - 1])$

20.3 Why It Belongs to Interval DP

It uses a substring range and depends on smaller substring ranges.

It is both:

- string DP
- interval DP

21. Full Problem Walkthrough 10 — Palindrome Partitioning II Review

21.1 State

Palindrome table:

isPal[l][r]

Minimum cuts:

dp[i] = minimum cuts for prefix s[0..i]

21.2 Why It Relates to Interval DP

The palindrome check itself is interval DP:

isPal[l][r] = s[l] == s[r] && isPal[l + 1][r - 1]

Then the final cut DP is 1D prefix DP.

Many problems combine interval DP with prefix DP.

22. Full Problem Walkthrough 11 — Boolean Parenthesization, Preview

22.1 Problem Summary

Given boolean symbols and operators:

T, F, &, |, ^

Count ways to parenthesize expression so it evaluates to true.

22.2 State

Need two values per interval:

trueWays[l][r]

falseWays[l][r]

22.3 Transition

Split at an operator k.

Combine left and right true/false counts depending on operator.

22.4 Lesson

Interval DP can store multiple values per range.

23. Full Problem Walkthrough 12 — Scramble String, Preview

23.1 Problem Summary

Check if one string is a scrambled version of another.

23.2 State

dp[len][i][j] = true if s1 substring starting at i of length len
can scramble to s2 substring starting at j of length len

23.3 Transition

Try split length k.

Two cases:

26. No swap

27. Swap

This is advanced interval DP over two strings.

24. Full Problem Walkthrough 13 — Remove Boxes, Preview

24.1 Problem Summary

Boxes have colors.

Remove consecutive boxes of same color.

Removing k boxes gives:

$k * k$

points.

Return maximum points.

24.2 Why Basic $dp[l][r]$ Is Not Enough

The score depends on how many same-colored boxes are connected from outside the interval.

Need extra state:

$dp[l][r][k]$

where k represents same-colored boxes attached to one side.

24.3 Lesson

If interval behavior depends on outside context, add another dimension.

25. Full Problem Walkthrough 14 — Stone Game II, Preview

25.1 Problem Summary

Players take piles from the front.

Can take up to $2M$ piles.

M changes after each move.

25.2 State

This is not classic $dp[l][r]$.

Better state:

$dp[i][M]$ = max stones current player can get starting at pile i with parameter M

25.3 Lesson

Not every range-looking game is interval DP.

Sometimes the right state is index + game parameter.

26. Full Problem Walkthrough 15 — Stone Game III, Preview

26.1 Problem Summary

Players take 1, 2, or 3 stones from the front.
Return winner.

26.2 State

This is 1D game DP:

$dp[i]$ = max score difference current player can achieve from i onward

26.3 Lesson

Use interval DP only when both ends or arbitrary range splits matter.
If taking happens only from the front, 1D suffix DP may be enough.

27. Common Interval DP Edge Cases

27.1 Empty Interval

Often:

$dp[l][r] = 0$

when $l \geq r$ or no items exist inside.

27.2 Single Element

Base case depends on problem:

- game DP: value of element
- palindrome DP: 1
- strange printer: 1
- split DP: 0

27.3 Two Elements

Important for boundary transitions.

27.4 No Split Point

For open intervals where no k exists:

$dp[l][r] = 0$

27.5 Large Costs

Use long long if multiplication or sums can be large.

28. Common Interval DP Mistakes

Mistake 1 — Wrong Fill Order

$dp[l][r]$ depends on shorter intervals, so fill by length.

Mistake 2 — Wrong Boundaries

Open interval:

k from $l + 1$ to $r - 1$

Closed interval:

k from l to r

Mistake 3 — Forgetting Sentinels

Burst Balloons needs boundary 1s.

Cut Stick needs boundary 0 and n .

Mistake 4 — Choosing First Operation When Last Is Easier

For Burst Balloons, choosing first is messy.

Choosing last fixes neighbors.

Mistake 5 — Incorrect INF Initialization

For min problems, initialize each interval before taking min.

Mistake 6 — Missing Feasibility Check

Merge Stones requires:

$(n - 1) \% (k - 1) == 0$

29. Interview Explanation Templates

29.1 General Interval DP

I define $dp[l][r]$ as the optimal answer for the interval from l to r . I compute shorter intervals first, then use them to build larger intervals.

29.2 Split Interval DP

For each interval $[l, r]$, I try every possible split point k . The answer combines the best result from the left interval, the right interval, and the cost caused by splitting or merging at k .

29.3 Burst Balloons

Instead of choosing which balloon to burst first, I choose which balloon is burst last in the interval. This keeps the left and right boundaries fixed.

29.4 Minimum Cost to Cut Stick

For a segment between two cut boundaries, I try every possible first cut. The cost of that cut is the segment length, plus the optimal costs of cutting the two resulting subsegments.

29.5 Game Interval DP

I store the maximum score difference the current player can achieve. Taking one side gives immediate score minus the opponent's best future difference.

30. Complexity Patterns

Burst Balloons	$O(n^3)$	$O(n^2)$
Minimum Cost to Cut Stick	$O(n^3)$	$O(n^2)$
Matrix Chain Multiplication	$O(n^3)$	$O(n^2)$
Polygon Triangulation	$O(n^3)$	$O(n^2)$
Stone Game	$O(n^2)$	$O(n^2)$
Stone Game VII	$O(n^2)$	$O(n^2)$
Merge Stones	$O(n^3)$	$O(n^2)$
Strange Printer	$O(n^3)$	$O(n^2)$
Longest Palindromic Subsequence	$O(n^2)$	$O(n^2)$
Palindrome Table	$O(n^2)$	$O(n^2)$

31. Time-Boxed Practice Guidance

Intro Interval DP

Target:

35–50 minutes

Problems:

- Longest Palindromic Subsequence
- Stone Game
- Minimum Score Triangulation

Medium Interval DP

Target:

45–75 minutes

Problems:

- Burst Balloons
- Minimum Cost to Cut a Stick
- Stone Game VII
- Strange Printer

Hard Interval DP

Target:

75–120 minutes

Problems:

- Merge Stones
- Remove Boxes
- Scramble String
- Boolean Parenthesization

- Count Different Palindromic Subsequences
-

32. Expanded Practice Set

32.1 Must-Do Medium

- 28. Longest Palindromic Subsequence
- 29. Stone Game
- 30. Stone Game VII
- 31. Minimum Score Triangulation of Polygon
- 32. Minimum Cost to Cut a Stick
- 33. Burst Balloons
- 34. Strange Printer

32.2 Strong Interview Practice

- 35. Matrix Chain Multiplication
- 36. Merge Stones
- 37. Palindrome Partitioning II
- 38. Scramble String
- 39. Predict the Winner
- 40. Minimum Insertion Steps to Make a String Palindrome

32.3 Hard / Stretch

- 41. Remove Boxes
 - 42. Merge Stones
 - 43. Count Different Palindromic Subsequences
 - 44. Boolean Parenthesization
 - 45. Strange Printer
 - 46. Scramble String
 - 47. Minimum Cost Tree From Leaf Values
-

33. Suggested Study Order

Step 1 — Simple Range DP

Start with:

- 48. Longest Palindromic Subsequence
- 49. Stone Game
- 50. Predict the Winner

Goal:

Understand $dp[l][r]$.

Step 2 — Split Point DP

Solve:

- 51. Matrix Chain Multiplication
- 52. Minimum Score Triangulation
- 53. Minimum Cost to Cut a Stick

Goal:

Learn split-point transitions.

Step 3 — Choose Last Operation

Solve:

- 54. Burst Balloons

Goal:

Master the “choose last” trick.

Step 4 — Hard Range Compression

Solve:

55. Strange Printer

56. Merge Stones

Goal:

Learn harder interval states and boundary logic.

Step 5 — Advanced Extra State

Solve:

57. Scramble String

58. Remove Boxes

Goal:

Learn when $dp[l][r]$ is not enough.

34. Pattern Decision Table

burst/remove changes neighbors	choose last operation	Burst Balloons
cut segment with cost length	try cut inside interval	Cut Stick
parenthesize expression	split at operator	Boolean Parenthesization
multiply chain	split matrix chain	Matrix Chain
triangulate polygon	choose middle vertex	Triangulation
take from either end	score difference DP	Stone Game
merge consecutive piles	split and merge interval	Merge Stones
palindrome range	shrink ends	LPS
repeated printing	merge same characters	Strange Printer
outside context matters	add extra dimension	Remove Boxes

35. Mini Problem 1 — Predict the Winner

```

class Solution {
public:
    bool PredictTheWinner(vector<int>& nums) {
        int n = nums.size();

        vector<vector<int>> dp(n, vector<int>(n, 0));

        for (int i = 0; i < n; i++) {
            dp[i][i] = nums[i];
        }

        for (int len = 2; len <= n; len++) {
            for (int l = 0; l + len - 1 < n; l++) {
                int r = l + len - 1;

                dp[l][r] = max(nums[l] - dp[l + 1][r],
                               nums[r] - dp[l][r - 1]);
            }
        }

        return dp[0][n - 1] >= 0;
    }
};

```

Pattern:

score difference from interval

36. Mini Problem 2 — Minimum Cost Tree From Leaf Values

Idea

Given array, build binary tree.

Each non-leaf node has value:

max leaf in left subtree * max leaf in right subtree

Interval DP version:

```

class Solution {
public:
    int mctFromLeafValues(vector<int>& arr) {
        int n = arr.size();
    }
};

```

```

vector<vector<int>> maxVal(n, vector<int>(n, 0));

for (int l = 0; l < n; l++) {
    maxVal[l][l] = arr[l];

    for (int r = l + 1; r < n; r++) {
        maxVal[l][r] = max(maxVal[l][r - 1], arr[r]);
    }
}

vector<vector<int>> dp(n, vector<int>(n, 0));

for (int len = 2; len <= n; len++) {
    for (int l = 0; l + len - 1 < n; l++) {
        int r = l + len - 1;

        dp[l][r] = 1e9;

        for (int k = l; k < r; k++) {
            int cost = dp[l][k]
                + dp[k + 1][r]
                + maxVal[l][k] * maxVal[k + 1][r];

            dp[l][r] = min(dp[l][r], cost);
        }
    }
}

return dp[0][n - 1];
}
};

```

Note:

There is also a greedy stack solution, but interval DP is useful for learning.

37. Mini Problem 3 — Palindrome Table

```

vector<vector<bool>> buildPalindromeTable(string& s) {
    int n = s.size();

    vector<vector<bool>> isPal(n, vector<bool>(n, false));

```

```

for (int len = 1; len <= n; len++) {
    for (int l = 0; l + len - 1 < n; l++) {
        int r = l + len - 1;

        if (s[l] == s[r]) {
            if (len <= 2 || isPal[l + 1][r - 1]) {
                isPal[l][r] = true;
            }
        }
    }
}

return isPal;
}

```

Used in:

- Palindromic Substrings
- Longest Palindromic Substring
- Palindrome Partitioning II

38. Mini Problem 4 — Generic Split DP Skeleton

```

for (int len = 2; len <= n; len++) {
    for (int l = 0; l + len - 1 < n; l++) {
        int r = l + len - 1;

        dp[l][r] = INF;

        for (int k = l; k < r; k++) {
            dp[l][r] = min(dp[l][r],
                           dp[l][k] + dp[k + 1][r] + cost(l, k, r));
        }
    }
}

```

Use this whenever:

interval answer = best split of left interval + right interval + merge cost

39. Common Edge Case Checklist

Before submitting interval DP code, test:

n = 0
n = 1
n = 2
all values equal
strictly increasing values
strictly decreasing values
no split point exists
single cut
no cut
palindrome length 1
palindrome length 2
large multiplication values
impossible merge condition

40. Debugging Interval DP

If interval DP fails:

59. Define $dp[l][r]$ exactly.
 60. Decide whether interval is open or closed.
 61. Check base cases.
 62. Check fill order by length.
 63. Verify split-point range.
 64. Check sentinel boundaries.
 65. Check if choosing last operation is needed.
 66. Check whether extra state is required.
 67. Print small DP table for small input.
-

41. What to Master Before Chapter 40

Before moving to Bitmask Dynamic Programming, make sure you can:

- Define $dp[l][r]$
 - Fill by interval length
 - Distinguish open vs closed intervals
 - Solve Longest Palindromic Subsequence
 - Solve Stone Game / Predict the Winner
 - Solve Minimum Cost to Cut a Stick
 - Solve Matrix Chain Multiplication
 - Solve Minimum Score Triangulation
 - Solve Burst Balloons
 - Understand the choose-last-operation trick
 - Understand why some interval DP needs extra state
 - Recognize $O(n^3)$ split-point transitions
-

42. Final Cheat Sheet

Fill by Length

```
for (int len = 1; len <= n; len++) {  
    for (int l = 0; l + len - 1 < n; l++) {  
        int r = l + len - 1;  
    }  
}
```

Split Point

```
for (int k = l; k < r; k++) {  
    dp[l][r] = min(dp[l][r],  
        dp[l][k] + dp[k + 1][r] + cost);  
}
```

Open Interval Split

```
for (int k = l + 1; k < r; k++) {  
    dp[l][r] = max(dp[l][r],  
        dp[l][k] + dp[k][r] + cost);  
}
```

Game Difference

```
dp[l][r] = max(nums[l] - dp[l + 1][r],  
    nums[r] - dp[l][r - 1]);
```

Palindrome Range

```
if (s[l] == s[r]) {  
    dp[l][r] = 2 + dp[l + 1][r - 1];  
} else {  
    dp[l][r] = max(dp[l + 1][r], dp[l][r - 1]);  
}
```

Burst Balloons

```
dp[l][r] = max(dp[l][r],  
    dp[l][k] + nums[l] * nums[k] * nums[r] + dp[k][r]);
```

43. Core Takeaway

Interval DP is about solving ranges.

The main state is:

```
dp[l][r]
```

The most common strategy is:

try every split point k

The most important trick is:

choose the last operation instead of the first operation

Use interval DP when:

- the problem is about subarrays/substrings/ranges
- operations split a range into smaller ranges
- costs depend on interval boundaries
- operation order affects neighbors

The usual fill order is:

short intervals first, long intervals later

Most interval DP problems are $O(n^3)$, because:

$O(n^2)$ intervals * $O(n)$ split points

Once you can define the interval and choose the right split point, interval DP becomes much more manageable.